

Google Career — Day 6 Detailed To-Do

Yes. I checked the current GitHub state first, so Day 6 is based on evidence rather than assuming your Day 5 work.

Your Project 1 repository has now moved forward: it shows 6 commits, includes migrations/, PostgreSQL connection/database files, and PostgreSQL integration tests. The README now documents:

HTTP Client → FastAPI Router → DocumentService → DocumentRepository → Database connection → PostgreSQL

and PostgreSQL-backed CRUD, migrations, connection lifecycle, and integration tests.

Your google-interview-prep repository also now contains day5/, alongside day1/, day2/, and day4/.

So Day 6 should NOT repeat Day 5 PostgreSQL implementation. It should verify, deepen, test, explain, and build interview-level mastery on top of it.

Day 6 Theme

PostgreSQL Verification + SQL Depth + Prefix Sum/Hashing + Backend System Design

Primary objective

By the end of Day 6, you should be able to:

1. Explain your PostgreSQL implementation without looking at the code.
2. Debug database failures deliberately.
3. Explain SQL CRUD, transactions, indexes and connection management.
4. Prove that Project 1 actually persists documents in PostgreSQL.
5. Understand the Prefix Sum + Hashing DSA pattern deeply.
6. Solve Subarray Sum Equals K.
7. Understand when Prefix Sum is better than Sliding Window.
8. Design a scalable document-storage backend.
9. Defend your architectural decisions in a Google-style interview.
10. Leave Project 1 in a clean, tested, documented state.

Day 6 Time Budget

Target: 3 hours

Activity Time
Dependency/checkpoint 10 min
PostgreSQL + SQL deep revision 35 min
Project 1 persistence verification 45 min
Database failure/debugging practice 20 min
DSA: Prefix Sum + Hashing 40 min
System Design 15 min
Interview practice 10 min
Documentation + Git 5 min

Do not expand this into another 5-6 hour study day.

The goal is engineering depth, not topic accumulation.

1. FIRST — Day 5 Dependency Check

Time: 10 minutes

Before learning anything new, verify the current state.

Your GitHub repository already shows PostgreSQL-related implementation, including:

- migrations/001_create_documents_table.sql
- src/db/connection.py
- src/database.py
- src/db/schema.sql
- PostgreSQL repository implementation
- tests/test_postgres_integration.py

The README also documents migration tracking through schema_migrations and PostgreSQL integration tests.

However, GitHub proves only what is committed.

It does NOT prove that your local environment works right now.

Checklist

Run your project locally.

Verify:

- ☐ PostgreSQL is running.
- ☐ Correct database exists.
- ☐ .env exists locally.
- ☐ .env is NOT committed.
- ☐ .env.example exists.
- ☐ DATABASE_URL is correct.
- ☐ Application starts.
- ☐ Database connection works.
- ☐ Migration runs successfully.
- ☐ documents table exists.
- ☐ API starts.
- ☐ PostgreSQL integration tests execute.
- ☐ Full pytest suite executes.

2. Verify Database Persistence

This is extremely important.

Do not merely run:

```
POST /documents
GET /documents
```

and conclude PostgreSQL works.

You need to prove persistence.

Experiment

Step 1

Create a document:

```
POST /documents
```

Example:

```
{
  "name": "engine-requirement",
  "document_type": "requirement",
  "version": "1.0"
}
```

Record its returned ID.

Step 2

Retrieve it:

```
GET /documents/{id}
```

Confirm it exists.

Step 3

Inspect PostgreSQL

Run:

```
SELECT *
```

FROM documents;

Confirm the row exists in PostgreSQL.

Step 4

Stop the FastAPI application.

Restart it.

Step 5

Retrieve the same document again.

```
GET /documents/{id}
```

Expected conclusion

The document should still exist.

That proves:

API

↓

Service

↓

Repository

↓

PostgreSQL

rather than:

API

↓

Python dictionary

This distinction is critical in an interview.

3. PostgreSQL Deep Revision

Time: 35 minutes

Do not read PostgreSQL from scratch.

Your goal today is to be able to explain what you built.

3.1 Relational Database Fundamentals

You must be comfortable explaining:

Database

A logical collection of structured data managed by a database system.

Table

A structured collection of rows and columns.

Your Project 1 table:

documents

Row

One document record.

Column

One attribute:

id

name

document_type

version

status

created_at

updated_at

3.2 Primary Key

Be able to answer:

“Why does documents.id need to be a primary key?”

Because each document needs a unique identifier.

Your current schema:

id VARCHAR(255) PRIMARY KEY

Understand:

- uniqueness
- identity
- lookup
- indexing implications
- constraint enforcement

3.3 NOT NULL

Your schema contains:

name VARCHAR(255) NOT NULL

document_type VARCHAR(255) NOT NULL

version VARCHAR(255) NOT NULL

status VARCHAR(255) NOT NULL

Be able to explain:

“Why should these fields be NOT NULL?”

Because the application expects them to exist for a valid document.

3.4 DEFAULT

Your schema has:

status VARCHAR(255) NOT NULL DEFAULT 'created'

and:

```
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

Understand why defaults belong at the database layer in addition to application logic.

3.5 SQL CRUD

You should manually execute each operation.

INSERT

```
INSERT INTO documents (...)
```

VALUES (...);

SELECT

```
SELECT *
```

FROM documents;

SELECT by ID

```
SELECT *
```

FROM documents

WHERE id = ...;

UPDATE

```
UPDATE documents
```

SET version = ...

WHERE id = ...;

DELETE

```
DELETE FROM documents
```

WHERE id = ...;

Your repository follows exactly this CRUD mapping.

4. Deliberate Database Failures

Time: 20 minutes

This is one of the most valuable Day 6 exercises.

Do not only test the happy path.

Create failures intentionally.

Failure 1 — Duplicate Primary Key

Try inserting the same ID twice.

Understand:

Database constraint

↓

Database error

↓

Repository

↓

Application handling

Ask yourself:

“Where should this error be translated?”

Failure 2 — Missing Required Field

Try:

```
INSERT INTO documents (...)
```

without a required column.

Understand the database-level NOT NULL failure.

Failure 3 — Invalid Data

Try inserting data that violates the expected type or constraints.

Understand:

Application validation

vs

Database validation

Failure 4 — Update Nonexistent Document

Run:

```
UPDATE documents
```

```
SET version = '2.0'
```

```
WHERE id = 'does-not-exist';
```

Ask:

“How does the repository determine that nothing was updated?”

This is a very important backend interview question.

Failure 5 — Delete Nonexistent Document

Same idea:

```
DELETE FROM documents
```

WHERE id = 'does-not-exist';

Understand how the application converts this into:

404 Not Found

Your current API contract documents 404 behavior for missing documents.

5. Understand Database Transactions

You do not need to build a transaction-heavy workflow today.

You need to understand it.

Study:

```
BEGIN
```

↓

Operation 1

↓

Operation 2

↓

```
COMMIT
```

versus:

```
BEGIN
```

↓

Operation 1

↓

Operation 2 fails

↓

```
ROLLBACK
```

Know ACID:

A — Atomicity

All operations succeed or none do.

C — Consistency

Database moves between valid states.

I — Isolation

Concurrent transactions should not incorrectly interfere.

D — Durability

Committed data survives failures according to the database's durability guarantees.

6. Understand Your Connection Lifecycle

Your repository now uses a dedicated connection boundary.

The current project documentation states that `database_connection()`:

- opens a connection
- yields it to the repository
- rolls back on failure
- closes it in finally

You need to be able to explain:

Request

↓

Repository

↓

Open connection

↓

Execute SQL

↓

Commit / rollback

↓

Close connection

Interview question

“Why shouldn't every repository method independently reinvent connection handling?”

Answer conceptually:

Because database lifecycle management should be centralized to improve:

- consistency
- cleanup
- error handling
- maintainability
- testability

7. Migration Understanding

Your project now uses:

migrations/
001_create_documents_table.sql

and:

src/database.py

The README says migrations are tracked through:

schema_migrations

so already-applied migrations are not repeatedly executed.

You need to understand:

Developer

↓

Migration file

↓

Migration runner

↓

Database

↓

schema_migrations

Be able to answer

“Why is a migration better than manually creating the table?”

Because the schema becomes:

- reproducible
- versioned
- reviewable
- deployable
- consistent across environments

8. Project 1 — Persistence Verification

Time: 45 minutes

This is the main engineering task of Day 6.

You are not adding a major feature.

You are proving that your current architecture works.

Task 8.1 — Verify Repository Boundary

Open:

src/repositories/document_repository.py

Understand every method.

You should be able to explain:

create

get_all

get_by_id

update

delete

without reading the implementation line-by-line.

Task 8.2 — Trace One Request

Take:

```
POST /documents
```

and trace it manually:

HTTP Request

↓

FastAPI Router

↓

Pydantic validation

↓

DocumentService

↓

DocumentRepository

↓

Database connection

↓

PostgreSQL

↓

Repository result

↓

Service

↓

API response

Write this flow down.

This becomes interview material.

9. Trace a GET Request

Do the same for:

```
GET /documents/{document_id}
```

Understand:

Router

↓

Service

↓

Repository

↓

SQL SELECT

↓

PostgreSQL

↓

row

↓

domain object

↓

response

10. Trace an UPDATE

Trace:

```
PUT /documents/{document_id}
```

Understand:

Request

↓

validation

↓

service

↓

repository

↓

```
UPDATE SQL
```

↓

database

↓

updated document

↓

response

Ask:

“What happens if the ID does not exist?”

Your answer should be:

Repository detects absence

↓

DocumentNotFoundError

↓

Global FastAPI exception handler

↓

HTTP 404

Your repository's documented API behavior preserves this contract.

11. Verify DELETE

Run:

```
DELETE /documents/{id}
```

Then:

```
GET /documents/{id}
```

Expected:

404

Then inspect PostgreSQL.

The row should actually be gone.

12. Restart Persistence Test

This is mandatory.

Procedure

1. Start PostgreSQL.
2. Start FastAPI.
3. Create document.
4. Retrieve document.
5. Stop FastAPI.
6. Start FastAPI again.
7. Retrieve document.
8. Confirm it still exists.

Definition of success

The document survives the application restart.

13. PostgreSQL Integration Tests

Your repository now documents a dedicated:

tests/test_postgres_integration.py

The documented tests cover:

- PostgreSQL is used when DATABASE_URL is valid.
- create → fetch
- multiple creates → list
- create → update → fetch
- create → delete → fetch
- missing document → 404

The test fixture truncates documents between tests.

Run:

```
python -m pytest -q tests/test_postgres_integration.py
```

Then:

```
python -m pytest -q
```

Use the Python executable/environment appropriate to your local setup.

14. Do NOT Fabricate Test Results

Record the actual result.

For example:

Full test suite:

X passed

Y failed

Z warnings

If there are failures:

Failure:

Cause:

Fix:

Retest:

Result:

Do not write:

“All tests passed”

unless you actually ran them.

This rule applies throughout the project.

15. DSA — Prefix Sum + Hashing

Time: 40 minutes

This is the major DSA focus of Day 6.

You have already learned:

- Two Pointers
- Sliding Window
- Prefix Sum basics

Today you need to understand the Prefix Sum + Hash Map combination.

16. Core Prefix Sum Concept

Given:

[2, 4, 1, 3]

Prefix sums:

[2, 6, 7, 10]

More formally:

$\text{prefix}[i] = \text{sum of elements from } 0 \text{ through } i$

17. The Critical Identity

For a subarray:

$i \dots j$

the sum is:

$\text{prefix}[j] - \text{prefix}[i - 1]$

Suppose we need:

$\text{sum} = k$

Then:

$\text{prefix}[j] - \text{prefix}[i] = k$

Therefore:

$\text{prefix}[i] = \text{prefix}[j] - k$

This is the heart of:

Prefix Sum + Hash Map

18. Why Hashing?

Suppose the current prefix sum is:

`current_sum`

We need to know whether:

`current_sum - k`

has appeared previously.

A hash map provides approximately:

$O(1)$

lookup.

Therefore:

$O(n)$

instead of:

$O(n^2)$

19. The Critical Initialization

You must understand:

```
prefix_counts = {0: 1}
```

Why?

Because a subarray beginning at index 0 may itself have sum k .

Example:

`[3, 1, 2]`

with:

$k = 3$

At the first element:

`prefix = 3`

We need:

$3 - 3 = 0$

The map must already contain:

0

This is one of the most frequently tested details of the pattern.

20. Day 6 DSA Problem

Subarray Sum Equals K

You must solve:

“Given an integer array `nums` and integer `k`, return the total number of subarrays whose sum equals `k`.”

Example:

Input:

[2, 1, 1]

`k = 2`

Output:

2

21. Required Learning Format

Do not merely memorize the code.

Write your solution using:

1. Problem

What are we asked to find?

2. Pattern

Prefix Sum + Hash Map

3. Intuition

Explain why:

$\text{current_prefix} - \text{previous_prefix} = k$

4. Brute Force

Consider every subarray.

Complexity:

$O(n^2)$

5. Optimized

Maintain:

`prefix_sum`

`frequency_map`

6. Complexity

Time: $O(n)$

Space: $O(n)$

7. Edge Cases

Test at least:

`[]`

`[1]`

`[1, -1]`

`[1, 2, 3]`

`[0, 0, 0]`

negative numbers

$k = 0$

subarray starts at index 0

repeated prefix sums

8. Mistakes

Explicitly document:

- forgetting `{0: 1}`
- storing only whether a prefix exists
- not storing frequency
- confusing prefix sum with subarray sum

22. VERY IMPORTANT — Why Frequency Matters

Suppose the same prefix sum appears multiple times.

You cannot simply store:

`seen.add(prefix_sum)`

You need:

`prefix_counts[prefix_sum]`

because one current prefix may form valid subarrays with multiple earlier prefixes.

This distinction is fundamental.

23. Compare the Three Patterns

By the end of Day 6, you should be able to distinguish:

	Pattern		Typical use	
	Two Pointers		Ordered/paired relationships	
	Sliding Window		Contiguous range with maintainable window condition	
	Prefix Sum + Hashing		Subarray sums/counts, especially with negatives	

The critical insight:

Sliding Window

Often relies on being able to adjust the window monotonically.

Prefix Sum + Hashing

Works even when negative numbers make ordinary sliding-window expansion/shrinking unsuitable.

24. DSA Interview Drill

After solving the problem, explain it aloud without code.

Answer:

“Why can't we always use Sliding Window for Subarray Sum Equals K?”

Then:

“Why does the presence of negative numbers matter?”

Then:

“Why do we store frequencies rather than a set?”

Then:

“Why initialize the map with {0: 1}?”

Then:

“Why is the algorithm $O(n)$?”

If you cannot answer these without looking at notes, the topic is not finished.

25. System Design

Time: 15 minutes

Do a small design exercise.

Design a Document Storage Service

This directly extends Project 1.

Requirements

Users should be able to:

- create documents
- retrieve documents
- list documents
- update documents
- delete documents

Scale

Start with:

1 million documents

Then ask:

“What changes at 10 million?”

Then:

“What changes at 100 million?”

APIs

Design:

```
POST /documents
GET /documents
GET /documents/{id}
PUT /documents/{id}
DELETE /documents/{id}
```

Data Model

Start with:

documents

id

name

document_type

version

status

created_at

updated_at

This corresponds to the current Project 1 schema.

Architecture

Start:

Client

↓

Load Balancer

↓

FastAPI instances

↓

Service Layer

↓

Repository

↓

PostgreSQL

Then think about:

Redis

Read replicas

Connection pooling

Object storage

Message queue

Partitioning

But do not implement these today.

26. System Design Questions

Answer these on paper:

Q1

What happens if PostgreSQL goes down?

Q2

How do you scale reads?

Q3

How do you scale writes?

Q4

When would you introduce Redis?

Q5

When would PostgreSQL become a bottleneck?

Q6

How would you partition 100M documents?

Q7

How would you handle concurrent updates?

Q8

How would you monitor the service?

Q9

What metrics would you collect?

Q10

What happens if a database connection leaks?

27. Interview Practice

Time: 10 minutes

Do a spoken mock interview.

You should answer these without notes.

Project

1. Explain your architecture.
2. Why did you introduce a repository layer?
3. Why not put SQL inside FastAPI routes?
4. Why is business logic in the service layer?
5. Why PostgreSQL?
6. How does a document get persisted?
7. How do you handle missing documents?
8. How do you handle database failures?
9. How do migrations work?
10. How do your integration tests work?

SQL

11. What is a primary key?
12. What is an index?
13. What is a transaction?
14. Explain ACID.
15. What is the difference between DELETE and TRUNCATE?
16. Why use NOT NULL?
17. Why use migrations?

DSA

18. Explain Prefix Sum.
19. Explain Prefix Sum + Hashing.
20. Why {0: 1}?

21. Why use a frequency map?

22. Why can negative numbers break a normal sliding-window approach?

28. Documentation

Time: 5 minutes

Update your interview-prep repository.

Current GitHub state shows:

day1/

day2/

day4/

day5/

notes/

Create:

day6/

├─ dsa/

| └─ prefix_sum_hashing.md

├─ databases/

| └─ postgres_deep_dive.md

├─ system_design/

| └─ document_storage.md

├─ interview/

| └─ day6_questions.md

└─ README.md

29. What to Put in prefix_sum_hashing.md

Include:

- Prefix Sum definition
- Prefix Sum formula
- Prefix Sum + Hash Map
- Mathematical derivation
- {0: 1} explanation
- Frequency-map explanation
- Subarray Sum Equals K
- brute force
- optimized approach
- complexity
- edge cases
- common mistakes

- Sliding Window comparison
- interview questions

30. What to Put in postgres_deep_dive.md

Include:

- relational database
- table/row/column
- primary key
- constraints
- NOT NULL
- defaults
- SQL CRUD
- transactions
- ACID
- connection lifecycle
- migrations
- repository pattern
- database failures
- integration testing

Tie every concept back to your Project 1.

31. What to Put in document_storage.md

Use:

Requirements

↓

Scale

↓

APIs

↓

Data Model

↓

Architecture

↓

Indexes

↓

Caching

↓

Partitioning

↓

Consistency

↓

Reliability

↓

Monitoring

↓

Cost

Do not just write generic definitions.

Use your actual document service as the starting point.

32. Project 1 README Verification

Your current README already documents:

- PostgreSQL architecture
- schema
- environment configuration
- migrations
- connection lifecycle
- API
- PostgreSQL CRUD
- API tests
- integration tests
- test execution

Therefore, do not rewrite this documentation unnecessarily.

Instead, check:

- ☐ Is the README accurate?
- ☐ Does it match the current code?
- ☐ Are all commands actually executable?
- ☐ Are there stale references to in-memory persistence?
- ☐ Does `.env.example` match the required configuration?
- ☐ Are migration instructions reproducible?
- ☐ Are test instructions correct?

Only modify what is actually wrong or missing.

33. Git Discipline

Day 6 is not about creating many commits.

If you make meaningful changes, use focused commits.

Possible commits:

test: verify postgres persistence behavior

test: strengthen postgres integration coverage

docs: add postgres and prefix sum interview notes

docs: add document storage system design

If no code changes are necessary, do not manufacture a commit.

A clean repository is more valuable than artificial commit volume.

34. Day 6 — Definition of Done

You are DONE with Day 6 only if all of the following are true.

Project 1

- ☐ PostgreSQL application starts successfully.
- ☐ Database connection works.
- ☐ Migration works.
- ☐ documents table exists.
- ☐ Create persists to PostgreSQL.
- ☐ Read retrieves from PostgreSQL.
- ☐ Update persists.
- ☐ Delete persists.
- ☐ Missing document produces 404.
- ☐ Application restart does not lose persisted data.
- ☐ PostgreSQL integration tests run.
- ☐ Full test suite runs.
- ☐ Actual test result recorded.
- ☐ Database failure behavior understood.

PostgreSQL Knowledge

You can explain without notes:

- ☐ Primary key
- ☐ NOT NULL
- ☐ Defaults
- ☐ SQL CRUD
- ☐ Transactions
- ☐ ACID
- ☐ Connection lifecycle
- ☐ Migrations
- ☐ Repository pattern
- ☐ Database constraints
- ☐ Integration testing

DSA

You can solve Subarray Sum Equals K from scratch.

- ☐ Recognize Prefix Sum + Hashing
- ☐ Derive the formula

- ☐ Explain {0: 1}
- ☐ Explain frequency counting
- ☐ Explain negative numbers
- ☐ Write brute force
- ☐ Write optimized solution
- ☐ Give complexity
- ☐ Handle edge cases
- ☐ Explain the solution verbally

System Design

You can design the document service at:

1M

10M

100M

and discuss:

- ☐ database
- ☐ indexes
- ☐ caching
- ☐ replicas
- ☐ partitioning
- ☐ consistency
- ☐ reliability
- ☐ monitoring
- ☐ failure handling
- ☐ cost

Interview

You can answer the 22 questions above without reading your notes.

35. What NOT To Do on Day 6

This is important.

☐ **Do not add Redis**

Not yet.

☐ **Do not add Docker**

Not yet.

☐ **Do not add Kubernetes**

Not yet.

☐ **Do not add GCP**

Not yet.

☐ **Do not add LangChain**

Not yet.

☐ **Do not add embeddings**

Not yet.

☐ **Do not build RAG**

Not yet.

☐ **Do not build agents**

Not yet.

☐ **Do not start React**

Not yet.

☐ **Do not redesign the entire backend**

Not necessary.

☐ **Do not solve 10 DSA problems**

One pattern mastered deeply is more valuable.

☐ **Do not blindly add database abstractions**

Your current repository is intentionally conservative; the README itself notes that the project avoids unnecessary over-engineering.

36. Day 6 Output Checklist

At the end of the day you should have:

PROJECT 1

└─ PostgreSQL persistence verified

└─ Integration tests verified

└─ Failure behavior tested

└─ README verified

GOOGLE-INTERVIEW-PREP

└─ day6/

└─ dsa/

| └─ prefix_sum_hashing.md

└─ databases/

- | └─ postgres_deep_dive.md
- |─ system_design/
- | └─ document_storage.md
- |─ interview/
- | └─ day6_questions.md
- └─ README.md

37. Day 6 → Day 7 Dependency

The progression should now be:

Day 1

Python + SWE foundations

↓

Day 2

FastAPI + layered backend

↓

Day 3

Revision

↓

Day 4

Backend architecture + Sliding Window

↓

Day 5

PostgreSQL persistence

↓

Day 6

PostgreSQL verification

+ Prefix Sum/Hashing

+ System Design

↓

Day 7

Next backend/DSA progression

The important milestone is that Project 1 has now crossed from an in-memory API into a persistent backend, while your DSA preparation has progressed from Two Pointers → Sliding Window → Prefix Sum + Hashing.

Your current GitHub state supports this progression: Project 1 now explicitly documents PostgreSQL-backed persistence, migrations and integration testing, while your interview-prep repository has a dedicated day5/ area ready for the next day of material.

Day 6 success criterion

Don't measure Day 6 by:

““I studied PostgreSQL and Prefix Sum.””

Measure it by:

““I can prove my backend persists data correctly, explain every layer and failure mode, solve Subarray Sum Equals K from first principles, and defend the architecture in a Google-style interview.””

That is the standard we should use going forward.

Revised Day 6 — DSA

You're right. Subarray Sum Equals K was already part of Day 5, so it should not be repeated on Day 6.

For Day 6, we'll keep the Prefix Sum + Hashing pattern but use new problems that expand your pattern recognition rather than repeating Day 5.

Pattern: Prefix Sum + Hashing / Prefix-Sum Applications

```
| # | Problem | Difficulty | Main Skill |  
|---|---|---|---|  
| 1 | Contiguous Array | Medium | Prefix sum transformation + HashMap |  
| 2 | Binary Subarrays With Sum | Medium | Prefix sum + frequency map |  
| 3 | Continuous Subarray Sum | Medium | Prefix sum + modulo + HashMap |
```

These are deliberately chosen because they teach three different variations of the same underlying idea.

Problem 1 — Contiguous Array

LeetCode 525

“Given a binary array, find the maximum length of a contiguous subarray with an equal number of 0 and 1.”

Example:

Input:

[0,1,0,1,1,0,0]

Output:

6

Pattern

Prefix Sum + HashMap

But the important transformation is:

0 → -1

1 → +1

Then the problem becomes:

“Find the longest subarray whose sum is 0.”

Key insight

If:

`prefix[j] == prefix[i]`

then:

$\text{prefix}[j] - \text{prefix}[i] = 0$

Therefore the subarray between them has equal numbers of +1 and -1.

What you must learn

- Why transform 0 $\rightarrow$ -1
- Why equal prefix sums identify a valid subarray
- Why we store the first occurrence
- Why we do NOT overwrite an earlier index
- Difference between counting subarrays and finding longest subarray
- Complexity: $O(n)$ time, $O(n)$ space

Important interview question

“Why do we store the earliest occurrence of a prefix sum?”

Because for the current index i , the earliest previous occurrence produces the longest possible subarray.

Problem 2 — Binary Subarrays With Sum

LeetCode 930

“Given a binary array and an integer `goal`, return the number of non-empty subarrays whose sum equals `goal`.”

Example:

Input:

```
nums = [1,0,1,0,1]
```

`goal = 2`

Output:

4

Pattern

Prefix Sum + Frequency HashMap

This looks similar to Subarray Sum Equals K, but the goal today is to understand the pattern without relying on the Day 5 problem.

Core idea

For current prefix:

`current_sum`

we need an earlier prefix:

`current_sum - goal`

So:

count += frequency[current_sum - goal]

What you must learn

- ☐ Prefix-sum derivation
- ☐ Frequency map
- ☐ Why multiple previous prefixes matter
- ☐ Why zeros create multiple valid subarrays
- ☐ Difference between set and frequency map
- ☐ Complexity analysis
- ☐ Edge cases when goal = 0

Critical example

[0,0,0]

For:

goal = 0

there are:

6

valid subarrays.

This is an excellent test of whether you actually understand frequency counting.

Problem 3 — Continuous Subarray Sum

LeetCode 523

“Given an integer array nums and integer k, determine whether there is a continuous subarray of at least two elements whose sum is a multiple of k.”

Example:

Input:

```
nums = [23,2,4,6,7]
```

k = 6

Output:

true

Pattern

Prefix Sum + HashMap + Modulo

This is the most important new variation for Day 6.

Instead of looking for:

$\text{prefix}[j] - \text{prefix}[i] = k$

we need:

$$(\text{prefix}[j] - \text{prefix}[i]) \% k = 0$$

Therefore:

$$\text{prefix}[j] \% k == \text{prefix}[i] \% k$$

Key insight

If two prefix sums have the same remainder modulo k , their difference is divisible by k .

Example:

$$\text{prefix}[i] = 14$$

$$\text{prefix}[j] = 26$$

$$k = 6$$

Both:

$$14 \% 6 = 2$$

$$26 \% 6 = 2$$

Therefore:

$$26 - 14 = 12$$

$$12 \% 6 = 0$$

So the subarray between them has a sum divisible by 6.

What you must learn

- ☐ Prefix sum modulo
- ☐ Why equal remainders matter
- ☐ HashMap storing earliest index
- ☐ Minimum subarray length constraint
- ☐ Special handling of $k = 0$
- ☐ Negative values/remainders
- ☐ Complexity

Day 6 DSA Execution

40-minute allocation

1. Contiguous Array — 15 min

Do:

Problem

→ Pattern

→ Brute Force

→ Transformation

→ Optimized intuition

→ Code

- Complexity
- Edge cases
- Mistakes

Do not look at the solution initially.

2. Binary Subarrays With Sum — 12 min

Focus on:

prefix_sum

+

frequency_map

Especially understand:

frequency[prefix_sum]

versus:

first_seen[prefix_sum]

These represent different problem requirements.

3. Continuous Subarray Sum — 13 min

Focus heavily on:

prefix_sum % k

and why:

same remainder

→ divisible difference

Day 6 Pattern Recognition Drill

After solving all three, close your notes and classify them:

Question 1

“Need the number of subarrays satisfying a sum condition?”

Think:

Prefix Sum + Frequency Map

Question 2

“Need the longest subarray satisfying a balance condition?”

Think:

Prefix Sum + Earliest Index Map

Question 3

“Need a subarray whose sum is divisible by k?”

Think:

Prefix Sum + Modulo + Earliest Index Map

Day 6 DSA Comparison

You should now have:

```
| Day | Pattern | Problems |
|---|---|---|
| Day 4 | Sliding Window | Longest Substring Without Repeating Characters, Maximum Average Subarray I, Minimum Size Subarray Sum |
| Day 5 | Prefix Sum | Your previously solved Prefix Sum problems |
| Day 6 | Prefix Sum + Hashing variations | Contiguous Array, Binary Subarrays With Sum, Continuous Subarray Sum |
```

So Day 6 is new problem-solving, not repetition.

Updated Day 6 Definition of Done — DSA

You are done only when you can solve all three without looking at solutions:

- ☐ Contiguous Array
- ☐ Binary Subarrays With Sum
- ☐ Continuous Subarray Sum

And for each:

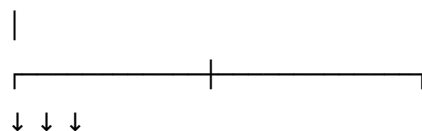
- ☐ Identify the pattern within ~2 minutes.
- ☐ Explain brute force.
- ☐ Derive optimized solution.
- ☐ Write code from scratch.
- ☐ Give time complexity.
- ☐ Give space complexity.
- ☐ Explain edge cases.
- ☐ Explain why the HashMap stores either frequency or earliest index.
- ☐ Explain the mathematical reason the approach works.
- ☐ State at least one common mistake.

Most important Day 6 goal

Don't memorize three solutions.

Learn to recognize:

Prefix Sum



Exact sum Balance Divisibility

| | |

frequency first index modulo + index

| | |

Binary Contiguous Continuous

Subarrays Array Subarray

With Sum

That is the pattern-recognition skill we're targeting for Google-style DSA interviews.